

LP-Pruned Quantum Branch-and-Bound for the 0/1 Knapsack Problem: Query-Complexity Analysis and Spectral Validation

Shiva Sai¹, Kumudini¹, Purusharth¹, Rakshit², Veerabhadra², C. Thirumal²

¹Student, IIT Bachupally, India

²Professor, Harvard University, USA

Abstract—The 0/1 knapsack problem is a fundamental NP-hard combinatorial optimization problem with applications spanning resource allocation, cryptography, and logistics. The best provable quantum algorithms either match classical meet-in-the-middle at $O(2^{n/2})$ or rely on unproven heuristic assumptions (Heuristic 2) and infeasible QRAQM hardware. We present a hybrid algorithm for knapsack that couples LP-guided reduced-cost variable fixing with Montanaro’s quantum branch-and-bound framework. Unlike prior quantum knapsack methods—which are generic, heuristic, or QRAQM-dependent—our method is simultaneously provable, gate-model feasible (QRAQM-free), knapsack-specific, and instance-adaptive. Our three-phase hybrid algorithm uses: (1) reduced-cost variable fixing to provably eliminate items from the search space, reducing the problem to $m = \alpha n$ core items where $\alpha \in [0, 1]$ depends on instance structure; (2) a classical threshold-search outer loop invoking quantum detection, with composed query complexity $O(\sqrt{T_{\max}} m \log m \log V_{\max})$, where $T_{\max}$ is the maximum LP-pruned tree size encountered over the tested thresholds; and (3) classical post-processing to reconstruct the global optimum. We prove instance-dependent complexity bounds showing that for weakly constrained instances, LP preprocessing reduces the effective problem size to near-constant, while for strongly constrained instances with substantially larger LP-pruned trees, the quantum phase achieves a quadratic query reduction relative to classical traversal of the same backtracking tree under the Montanaro oracle-model assumptions. Beyond complexity counting, we computationally test the spectral conditions for the quantum phase on the LP-pruned tree. We verify the effective spectral-gap inequality and phase-zero weight bound, finding perfect agreement with the analytical detection criterion on 889 completed simulated trees ($T_{\text{LP}} \leq 2000$). We benchmark against strong classical solvers (dynamic programming, Google OR-Tools), clarifying that the query-complexity reduction is relevant only in the large-coefficient, LP-hard regime where DP is infeasible and branch-and-bound explodes. A fault-tolerant resource estimate for our answer-agnostic reversible oracle (31–45 logical qubits) establishes baseline component costs for the predicate V . Finally, we characterize the NISQ noise boundary of amplitude amplification on IBM’s `ibm_fez` processor (156-qubit Heron r2) for core sizes $n \in \{3, 4, 5, 6\}$. We do not claim wall-clock quantum speedup, hardware advantage, or complete end-to-end full-walk gate complexity reduction.

Index Terms—quantum computing, knapsack problem, branch-and-bound, quantum walk, LP relaxation, variable fixing, combinatorial optimization

I. INTRODUCTION

The 0/1 knapsack problem is one of the most studied problems in combinatorial optimization: given n items, each

with a weight w_i and a value v_i , and a capacity W , find the subset that maximizes total value without exceeding the capacity. Despite its simple formulation, the problem is NP-hard, and the best classical exact algorithm—the Horowitz–Sahni meet-in-the-middle approach—runs in $O(2^{n/2})$ time and space [1].

Quantum computing has long promised speedups for combinatorial problems. Grover’s algorithm [2] provides a quadratic speedup for unstructured search, and its generalization via amplitude amplification [3] is a powerful tool. However, applying Grover search directly to knapsack yields $O(2^{n/2})$ —no improvement over classical meet-in-the-middle.

More sophisticated quantum approaches based on quantum walks have achieved better asymptotic complexities. Bernstein et al. [4] achieved $O(2^{0.241n})$, and Helm and May [5] further improved this to $O(2^{0.226n})$. However, these results rely on Heuristic 2—an unproven assumption about quantum walk update times—which Bonnetain et al. [6] have shown to be mathematically unsound in general. Furthermore, these algorithms require QRAQM (Quantum Random Access Memory with Quantum Addressing), a hardware model that does not exist and faces severe engineering challenges [7].

This creates a gap in the literature: can LP structure be combined with provable quantum tree-search query bounds without unproven heuristics or QRAQM?

A. Contributions and Novelty

Our algorithm uniquely combines exact LP reduced-cost fixing with a quantum tree search. Our defensible contributions are:

- 1) Exact LP/reduced-cost variable fixing for reducing the knapsack core before quantum tree search.
- 2) A threshold-optimization composition using a classical binary-search outer loop and a Montanaro-style quantum detection primitive.
- 3) A composed query bound $O(\sqrt{T_{\max}} m \log m \log V_{\max})$, with $T_{\max}$ explicitly defined as the maximum threshold-dependent LP-pruned tree size encountered by the outer loop.
- 4) Computational spectral validation of the implemented walk on 889 completed simulated trees with $T_{\text{LP}} \leq 2000$, including effective spectral-gap inequality, phase-

zero marked weight, and analytical phase-estimation acceptance separation.

- 5) Small-scale IBM hardware characterization of the reversible threshold predicate/core circuit, explicitly limited to the executed small instances.

We do not claim wall-clock quantum acceleration or a complete gate-complexity bound for the full LP-bounded walk, because the reversible LP heuristic h is not synthesized.

The remainder of this paper is organized as follows. Section II reviews related work. Section III establishes mathematical preliminaries. Section IV presents our algorithm. Section V provides the theoretical analysis. Section VI describes our experimental methodology and results. Section VII discusses implications and limitations. Section VIII concludes.

II. LITERATURE REVIEW

Classical exact methods. Dantzig’s LP relaxation yields a closed-form fractional optimum and the tight Dantzig upper bound that remains the workhorse of exact knapsack solvers [8]. The Horowitz–Sahni meet-in-the-middle (MitM) algorithm splits the items into two halves and merges sorted partial sums, attaining $O(2^{n/2})$ time and space [1]—still the best *provable* worst-case bound for exact 0/1 knapsack and the natural classical target for any quantum method. Kellerer, Pferschy, and Pisinger consolidate dynamic programming (pseudo-polynomial $O(nW)$), greedy bounds, and branch-and-bound (B&B) [9]. Pisinger’s *core* concept is central to our approach: empirically only items whose efficiency lies near the LP split affect the optimum [10], and reduced-cost variable fixing turns this observation into an *exact* reduction. Closely related subset-sum work includes the representation technique of Becker, Coron, and Joux [11] and its refinements [12], while Plateau and Elkihel analyze packing-tree pruning that prefigures modern B&B [13].

Quantum search and its limits. Grover’s algorithm searches an unstructured space of size N in $O(\sqrt{N})$ [2], later sharpened to exact zero-failure search [14] and generalized to amplitude amplification [3], with optimization variants via pure adaptive search [15] and Dürr–Høyer minimum finding [16]. Applied naively to knapsack, however, Grover yields only $O(2^{n/2})$ —no asymptotic gain over MitM—and Bennett et al. proved that black-box quadratic speedups cannot break the $2^{n/2}$ barrier for NP-complete problems absent additional structure [17]. Ambainis’ adversary method supplies matching query lower bounds [18], framing precisely what a structure-exploiting method must overcome.

Quantum walks and the heuristic/QRAQM barrier. Structured search improvements arise from quantum walks [19], formalized by the MNRS framework [20]. For subset-sum, Bernstein et al. reach $O(2^{0.241n})$ [4] and Helm–May $O(2^{0.226n})$ [5]—the strongest asymptotics known. Yet two caveats motivate our work: (i) they assume *Heuristic 2*, an unproven claim about walk-update costs that Bonnetain et al. showed to be mathematically unsound in general, recovering provable but weaker bounds [6]; and (ii) they require QRAQM (quantum-addressable quantum memory), whose feasibility is heavily

contested [7], [21]. The best “quantum” knapsack asymptotics are therefore simultaneously unproven and hardware-infeasible.

Implementable quantum knapsack and hybrids. A second line prioritizes constructibility on gate-model hardware. Grover-based knapsack solvers [22], [23] and quadratic/multidimensional extensions [24] are implementable but do not surpass MitM. Variational methods—QAOA [25], Grover adaptive search [26], and energy-systems applications [27]—run on NISQ devices but lack performance guarantees. Quantum-inspired metaheuristics (genetic [28], wolf-pack [29], harmonic-oscillator [30]) report practical gains without provable query reduction. The most relevant provable, QRAQM-free tool is Montanaro’s query reduction of B&B [31], building on quantum backtracking [32]; Sanavio et al. apply hybrid B&B to integer linear programs [33], and Wilkening et al.’s Quantum Tree Generator (QTG) builds superpositions of feasible knapsack solutions for QAOA [34]. Hardware context is set by Preskill’s NISQ analysis [35], shallow-circuit capabilities [36], and the Qiskit toolchain [37].

Closest prior work and our position. Three works are nearest to ours. Montanaro’s quantum B&B [31] is provable and QRAQM-free but *generic*: it bounds a quantum walk on an abstract tree of size T by $\tilde{O}(\sqrt{T}d)$ without analyzing any specific problem, the role of LP bounds, or the induced tree size for knapsack. The QTG of Wilkening et al. [34] is knapsack-specific and gate-model, but feeds feasible-solution superpositions to a *variational* (QAOA) routine and performs no LP-based variable fixing, inheriting QAOA’s lack of guarantees. Bonnetain et al. [6] restore provability to walk-based subset-sum yet still assume QRAQM. Our algorithm uniquely combines: (a) *exact* LP reduced-cost fixing with Montanaro’s provable, QRAQM-free quantum B&B specifically for knapsack, (b) an analysis of the resulting LP-pruned tree size $T_{\text{LP}} \leq T$, and (c) computational spectral validation of the explicit walk operator on completed simulable trees. Table I contrasts these properties.

The upper block of Table I shows that no single prior method is simultaneously provable, gate-model feasible, and knapsack-specific. The lower block isolates the capability our work contributes: an algorithm that couples *exact* LP variable fixing to a provable, QRAQM-free quantum walk, is *instance-adaptive*—with query complexity adapting to the LP-pruned tree sizes encountered—and whose spectral conditions and analytical detection criterion are computationally tested rather than assumed. This combination is what distinguishes the present work, and it is what we substantiate empirically in Section VI.

III. PRELIMINARIES

A. Problem Definition

The 0/1 Knapsack Problem is defined as follows. Given a set of n items, where item i has weight $w_i \in \mathbb{Z}^+$ and value

TABLE I
CAPABILITY COMPARISON WITH THE CLOSEST PRIOR QUANTUM
METHODS. THE LOWER BLOCK LISTS CAPABILITIES THAT, TO OUR
KNOWLEDGE, NO PRIOR METHOD PROVIDES SIMULTANEOUSLY.

Capability	Mont. [31]	QTG [34]	Bonn. [6]
Provable query reduction	Yes	No [§]	Yes
Gate-model feasible (no QRAQM)	Yes	Yes	No
Knapsack-specific	No	Yes	No
Exact LP variable fixing	No	No	No
Instance-adaptive [†]	No	No	No
Explicit walk spectral validation [‡]	No	No	No
Hardware demonstration	No	No	No

Mont.: Montanaro B&B; QTG: Quantum Tree Generator; Bonn.: Bonnetain et al.

[†]Query bound depends on the LP-pruned tree sizes encountered.

[‡]Explicit walk spectra and analytical detection criteria are evaluated computationally on classical branch-and-bound algorithms.

[§]Variational (QAOA): no performance guarantee.

$v_i \in \mathbb{Z}^+$, and a capacity $W \in \mathbb{Z}^+$, find a binary vector $x^* = (x_1^*, x_2^*, \dots, x_n^*) \in \{0, 1\}^n$ that solves

$$\max \sum_{i=1}^n v_i x_i \quad \text{subject to} \quad \sum_{i=1}^n w_i x_i \leq W. \quad (1)$$

We denote the optimal value by $Z^* = \sum_i v_i x_i^*$. Without loss of generality, we assume $w_i \leq W$ for all i (items heavier than the capacity are trivially excluded) and that all efficiencies $e_i = v_i/w_i$ are distinct.

B. LP Relaxation, Split Item, and Dantzig Bound

The LP relaxation of (1) permits fractional assignments $x_i \in [0, 1]$. Its solution has a closed-form structure due to Dantzig [8].

Definition 1 (Efficiency Ordering). Order items so that $e_1 > e_2 > \dots > e_n$ where $e_i = v_i/w_i$.

Definition 2 (Split Item). The *split item* s is the smallest index such that

$$\sum_{i=1}^s w_i > W \quad \text{and} \quad \sum_{i=1}^{s-1} w_i \leq W. \quad (2)$$

The LP optimal solution is then

$$x_i^{LP} = \begin{cases} 1 & \text{if } i < s, \\ (W - \sum_{j=1}^{s-1} w_j)/w_s & \text{if } i = s, \\ 0 & \text{if } i > s, \end{cases} \quad (3)$$

and the *Dantzig bound* is $Z_{LP} = \sum_{i=1}^n v_i x_i^{LP}$. This is computable in $O(n)$ time after sorting, and satisfies $Z^* \leq Z_{LP}$.

Definition 3 (Core Items). Given a parameter $\delta > 0$, the *core* C is the set of items with efficiencies close to the split item:

$$C = \{i : |e_i - e_s| \leq \delta \cdot e_s\}. \quad (4)$$

The remaining items are partitioned into *fixed-in* items $F_1 = \{i : e_i > e_s + \delta \cdot e_s\}$ and *fixed-out* items $F_0 = \{i : e_i < e_s - \delta \cdot e_s\}$.

The core concept was introduced by Pisinger [10], who showed empirically that $|C| \ll n$ for typical instances. Our algorithm exploits this structure theoretically.

C. Amplitude Amplification

Ours
We use the following standard result [3].

Theorem (Brassard et al., 2002). Let $\mathcal{A}$ be a quantum algorithm that produces $\mathcal{A}|0\rangle = \sin \theta |\Psi_{\text{good}}\rangle + \cos \theta |\Psi_{\text{bad}}\rangle$. Then amplitude amplification finds a good state in $O(1/\sqrt{p})$ oracle calls where $p = \sin^2 \theta$.

D. Quantum Branch-and-Bound (Montanaro)

Montanaro [31] proved that quantum computers can speed up classical branch-and-bound algorithms:

Theorem (Montanaro, 2020). Let $\mathcal{B}$ be a classical B&B algorithm that determines if there is a valid assignment evaluating to at least τ , exploring a tree of size T and maximum depth d . There is a quantum algorithm that solves the same detection problem with failure probability $\leq 1/3$ using

$$O(\sqrt{T} d \log d) \quad (5)$$

queries to the branching and bounding heuristics.

The algorithm uses a quantum walk on the implicit B&B tree and does not require QRAQM—only the ability to evaluate the bounding function (our LP relaxation) as a quantum oracle. This is the key quantum ingredient in our approach.

E. Reduced-Cost Variable Fixing

Given a lower bound Z_{LB} (e.g., from a greedy solution), an item i with $x_i^{LP} = 1$ can be *provably fixed* to 1 if removing it causes the LP upper bound to drop below Z_{LB} . Similarly, an item j with $x_j^{LP} = 0$ can be fixed to 0 if forcing it in causes the LP bound to drop below Z_{LB} . This technique, due to Pisinger [10], is exact—no heuristic assumptions are involved.

F. Meet-in-the-Middle Framework

The Horowitz–Sahni algorithm [1] partitions the n items into two disjoint sets A and B with $|A| = |B| = n/2$. It enumerates all $2^{n/2}$ subsets of A , storing pairs $(w(S_A), v(S_A))$ for each subset $S_A \subseteq A$, where $w(S_A) = \sum_{i \in S_A} w_i$ and $v(S_A) = \sum_{i \in S_A} v_i$. It does the same for B . Then, for each subset S_A of A , it finds the best compatible subset S_B of B satisfying $w(S_A) + w(S_B) \leq W$ using binary search on the sorted list of B -subsets. The total time and space are both $O(2^{n/2})$.

IV. PROPOSED ALGORITHM

Our hybrid algorithm operates in three phases: classical LP preprocessing, quantum branch-and-bound over the reduced core, and classical post-processing.

Algorithm 1 Reduced-Cost Variable Fixing**Require:** Items $(w_i, v_i)_{i=1}^n$, capacity W **Ensure:** Partition (F_1, C, F_0) , residual capacity W'

```

1: Compute efficiencies  $e_i \leftarrow v_i/w_i$ ; sort decreasingly
2: Compute greedy lower bound  $Z_{LB}$ 
3: Compute Dantzig upper bound  $Z_{LP}$ 
4: for each item  $i \in \{1, \dots, n\}$  do
5:   if  $x_i^{LP} = 1$  (item before split) then
6:      $Z'_i \leftarrow$  LP bound with item  $i$  forced out
7:     if  $Z'_i < Z_{LB}$  then
8:        $F_1 \leftarrow F_1 \cup \{i\}$  {Provably in optimal}
9:     else
10:       $C \leftarrow C \cup \{i\}$ 
11:    end if
12:  else
13:     $Z'_i \leftarrow$  LP bound with item  $i$  forced in
14:    if  $Z'_i < Z_{LB}$  then
15:       $F_0 \leftarrow F_0 \cup \{i\}$  {Provably out of optimal}
16:    else
17:       $C \leftarrow C \cup \{i\}$ 
18:    end if
19:  end if
20: end for
21:  $W' \leftarrow W - \sum_{i \in F_1} w_i$ ;  $V_1 \leftarrow \sum_{i \in F_1} v_i$ 
22: return  $(F_1, C, F_0)$ ,  $W'$ ,  $V_1$ 

```

A. Phase 1: LP Preprocessing via Reduced-Cost Variable Fixing

The classical preprocessing exploits the LP relaxation and reduced-cost analysis to provably partition items into three disjoint sets.

Unlike heuristic delta-based filtering, reduced-cost fixing is *provably exact*: no item in F_1 or F_0 can change the optimal solution. Let $m = |C|$ and $\alpha = m/n$.

B. Phase 2: Quantum Branch-and-Bound on Core

The core contribution of this paper. We construct a classical B&B tree over the m core items using LP relaxation bounds at each node, then apply Montanaro's quantum walk [31] to search this tree in

$$O\left(\sqrt{T_{\max}} m \log m \log V_{\max}\right)$$

queries.

1) *Classical B&B Tree Structure*: At each node of the B&B tree, we branch on a core item c_i (include/exclude) and compute the LP relaxation bound on the remaining subproblem. A node is pruned when its LP bound falls below the current best known integer solution.

Bounding oracle. The LP relaxation for the residual knapsack at each B&B node is computable in $O(m)$ time classically. The LP relaxation defines the bounding heuristic h required by the abstract query model.

2) *Reversible LP-Bound Oracle (h)*: To implement the bounding heuristic h as a quantum oracle, we construct a fixed-precision integer circuit that evaluates the exact threshold rule from Lemma V-A. The fixed-precision calculation evaluates a scaled integer bound $\tilde{Z}_{scaled} = V_{int}2^k + W_{res}\lfloor v_s 2^k / w_s \rfloor$. Because $\tilde{Z}_{scaled}$ is guaranteed to be an integer, the condition $\tilde{Z}'_i \leq Z_{LB} - 1/W_{max}$ is evaluated by comparing $\tilde{Z}_{scaled}$ against a classically pre-computed threshold $T = \lfloor Z_{LB}2^k - 2^k/W_{max} \rfloor$ without any precision loss.

The register layout allocates quantum memory for the scaled dividend, divisor, quotient, remainder, residual capacity, and the intermediate additions and multiplications. The arithmetic relies on the Thapliyal restoring divider [38] for calculating the fixed-point efficiency, the Vedral-Barenco-Ekert (VBE) multiplier [39] for computing the residual product, and the zero-ancilla Draper QFT adder [40]. By mapping the division remainder to the ancillary register required by Thapliyal's in-place division, the ancillary overhead is strictly bounded. Resource estimates for this layout are provided in Table II. Note that while the exact logical and ancillary qubit counts are established, full gate-level decomposition of the arithmetic blocks is deferred.

TABLE II

RESOURCE ESTIMATES FOR THE LP-BOUND BOUNDING ORACLE (h) AT VARYING CORE SIZES (m). THE (m, W_{max}) PAIRINGS SHOWN (15/31/63) ARE REPRESENTATIVE ILLUSTRATIVE EXAMPLE VALUES CHOSEN FOR THE DEMONSTRATION, NOT A DERIVED OR ASSUMED FUNCTIONAL RELATIONSHIP BETWEEN CORE SIZE AND MAXIMUM ITEM WEIGHT — THESE ARE INDEPENDENT PARAMETERS IN GENERAL KNAPSACK INSTANCES. QUBIT COUNTS ARE DERIVED FROM THE REGISTER LAYOUT MAPPING TO THE DRAPER, VEDRAL-BARENCO-EKERT, AND THAPLIYAL REVERSIBLE ARITHMETIC BLOCKS. THE FRACTIONAL FIXED-POINT PRECISION (k) SCALES WITH W_{max} PER LEMMA V-A. DEPTH, GATE, AND T-COUNTS ARE NOT YET AVAILABLE AS FULL GATE-LEVEL DECOMPOSITION HAS BEEN DEFERRED FOR THIS PREDICATE.

$m (W_{max})$	Precision k	Logical Qubits	Ancilla Qubits	Total Qubits	
4 (15)	8	81	1	82	N/A
6 (31)	10	101	1	102	N/A
8 (63)	12	121	1	122	N/A

3) *Quantum Walk Application*: We apply Montanaro's quantum B&B framework [31] to the implicit tree defined by our LP-bounded branching strategy:

Algorithm 2 Classical Threshold-Search Outer Loop (Phase 2)**Require:** Core items C with $|C| = m$, residual capacity W' , lower bound V_1 **Ensure:** Optimal core assignment x_C^*

```

1: Initialize binary search interval for threshold  $\tau$ 
2: while search interval  $> 0$  do
3:   Define LP-pruned fixed-threshold tree for  $\tau$ 
4:   Invoke quantum detection on that tree
5:   Update binary-search interval from the detection result
6: end while
7: Recover the optimum threshold/value after the loop
8: return Optimal core assignment  $x_C^*$ 

```

For each tested threshold τ_i , the fixed-threshold detection primitive requires $O(\sqrt{T_{\tau_i}} m \log m)$ queries under the Montanaro oracle-model assumptions. Since the classical outer loop tests at most $O(\log V_{\max})$ thresholds, summing these calls gives $O(\sqrt{T_{\max}} m \log m \log V_{\max})$ queries, where $T_{\max} = \max_i T_{\tau_i}$.

4) *Reversible Knapsack Oracle (Answer-Agnostic)*: The quantum subroutines require a constructible oracle that marks states satisfying the threshold predicate $\text{feasible}(x) \wedge \text{value}(x) \geq \tau$, where τ is the binary-search threshold—*never* the (unknown) optimal value. We implement this as a genuine gate-level reversible circuit using in-place Draper (QFT) arithmetic with *zero ancilla* beyond two accumulator registers: (1) accumulate $\sum_i w_i x_i$ into a weight register and subtract $W+1$, so its two's-complement sign bit equals feasibility; (2) accumulate $\sum_i v_i x_i$ and subtract τ , so its sign bit encodes value $\geq \tau$; (3) flip the phase conditioned on both flags; (4) uncompute.

Table III details the exact logical resource requirements for this evaluation/threshold predicate V . Crucially, we synthesize the predicate V only; the LP bounding heuristic h is not synthesized to a reversible Clifford+T circuit.

TABLE III
EXACT LOGICAL PREDICATE RESOURCES AND ANALYTICAL T-COUNT ESTIMATES

Core Size (m)	Qubits	Depth	Total Gates	2Q Gates	T-Count Estimate [†]
4	19	266	1353	573	$\approx 188,001$
6	22	300	1788	773	$\approx 227,631$
8	25	327	2243	977	$\approx 250,161$

[†]Heuristic analytical estimate assuming Ross-Selinger arbitrary R_z synthesis at $\epsilon = 10^{-13}$. The LP heuristic h was NOT synthesized.

This construction uses only the problem data and τ , so the optimum is *discovered* by the search rather than supplied to it—unlike a diagonal oracle built from a precomputed solution. We verified by exhaustive statevector simulation that, for all thresholds, the oracle marks exactly the brute-force-correct set and returns ancillae to $|0\rangle$, and that one amplitude-amplification stage with the full reversible circuit matches the verified phase action to $< 10^{-13}$. In a separate small-scale component experiment, a Dürr–Høyer-style maximum-finding loop built on this oracle recovered the exact optimum on the tested $n \leq 5$ statevector instances. This experiment is not the optimization loop used by quantum_bb_optimize.

C. Phase 3: Classical Post-Processing

Algorithm 3 Classical Reconstruction

Require: (F_1, C, F_0) , quantum solution x_C^* , capacity W

Ensure: Globally optimal solution x^*

1: $x^* \leftarrow F_1 \cup \{i \in C : x_{C,i}^* = 1\}$

2: $Z^* \leftarrow \sum_{i \in x^*} v_i$

3: **return** x^*, Z^*

Since reduced-cost fixing is exact, the combined solution $F_1 \cup x_C^*$ is globally optimal. No swap verification is needed.

D. Overall Algorithm

Algorithm 4 LP-Pruned Quantum B&B Knapsack Solver

Require: Items $(w_i, v_i)_{i=1}^n$, capacity W

Ensure: Optimal solution x^* and value Z^*

```

1: Phase 1:  $(F_1, C, F_0), W', V_1 \leftarrow \text{RC-FIX}(w, v, W)$ 
   {Alg. 1}
2:  $m \leftarrow |C|$ 
3: if  $m = 0$  then
4:   return  $x^* = F_1, Z^* = V_1$ 
5: else if  $m \leq m_{\text{thresh}}$  then
6:   Solve core classically via B&B
7: else
8:   Phase 2:  $x_C^* \leftarrow \text{QUANTUM-B\&B}(C, W', V_1)$  {Alg. 2}
9: end if
10: Phase 3:  $x^* \leftarrow F_1 \cup x_C^*$ 
11: return  $x^*, Z^* = V_1 + v_C(x_C^*)$ 

```

E. Algorithmic Extensions

1) *Multidimensional Knapsack Extension*: Our framework extends cleanly to the d -dimensional 0/1 knapsack problem. We generalize the LP fixing by using a surrogate relaxation: the minimum of d single-constraint Dantzig bounds. Sorting items by per-constraint efficiencies v_i/w_{ij} yields provably tight lower and upper bounds for reduced-cost fixing. Table IV summarizes the empirical growth of α for strongly correlated instances (e.g., $n = 24$). Because multidimensional instances increasingly resist classical variable fixing ($\alpha \rightarrow 1$), the observed increase in core ratio suggests that less work is removed by Phase 1 as dimensionality increases. The consequences for quantum query complexity require a separate analysis of the resulting pruned trees.

TABLE IV
CORE SIZE RATIO α VS. DIMENSIONALITY d ($n = 24$)

Dimensionality	$d = 1$	$d = 2$	$d = 3$	$d = 4$
Core Ratio α	0.28	0.51	0.72	0.83

V. THEORETICAL ANALYSIS

A. Correctness

Lemma 1a (Precision requirement for robust reduced-cost fixing). Assume a 0/1 knapsack instance with capacity W , max weight $W_{\max}$, and a greedy integer lower bound Z_{LB} . Let Z'_i be the exact perturbed LP bound for item i . If the LP bound is computed using k -bit floor truncation for the split item's efficiency e_s , yielding a truncated bound $\hat{Z}'_i$, the quantum detection loop will produce the **exact same** variable fixing partition (F_1, C, F_0) as infinite-precision classical Algorithm 1, provided two conditions are met:

- 1) The classical check $Z'_i < Z_{LB}$ is replaced by the robust check $\tilde{Z}'_i \leq Z_{LB} - 1/W_{max}$.
- 2) The precision satisfies $k > 2 \log_2(W_{max})$.

Note: The adoption of the robust check $\tilde{Z}'_i \leq Z_{LB} - 1/W_{max}$ entirely eliminates the exception for degenerate instances ($Z'_i = Z_{LB}$). Because the maximum truncation error $W_{max}2^{-k}$ is strictly less than the safety margin $1/W_{max}$, the truncated bound will never falsely cross the threshold, ensuring unconditional exactness for all instances.

Theorem 1. *Algorithm 4 returns the optimal solution to the 0/1 knapsack problem with probability at least $1 - 2^{-r}$ after r independent repetitions of the quantum detection phase, for all instances.*

Proof. We show that no optimal item is permanently excluded by the filtering phase.

Consider any optimal solution x^* and any item j with $x_j^* =$

1. There are three cases:

- 1) If $j \in F_1$, it is included in the solution during Phase 1.
- 2) If $j \in C$, it is considered in Phase 2, which searches the B&B tree over all 2^m subsets of the core.
- 3) The case $j \in F_0$ cannot occur: by the definition of reduced-cost fixing (Algorithm 1), if $j \in F_0$ then forcing j into the solution causes the LP upper bound to drop below the greedy lower bound $Z_{LB} \leq Z^*$. Since Z^* is the optimal value and the LP bound is an upper bound, this implies no optimal solution includes j .

For case (2), the algorithm tests thresholds via a classical binary-search outer loop. At each threshold, it invokes Montanaro’s quantum detection primitive, which correctly identifies if a valid assignment exceeding the threshold exists with high probability. By bounding the failure probability at each of the $O(\log V_{max})$ steps, the binary search converges to the optimal core assignment with high probability.

Since F_1 items are provably optimal, F_0 items are provably non-optimal, and the quantum phase finds the optimal core assignment, the combined solution $F_1 \cup x_C^*$ is globally optimal. Repeated applications with independent randomness yield success probability $1 - 2^{-r}$ after r independent runs. ■

Proposition 1 (Composed Query Complexity). *The implemented threshold-search composition algorithm solves the 0/1 knapsack problem using a classical outer loop to test thresholds. This implementation composes a classical binary-search outer loop with the fixed-threshold detection primitive, requiring at most $O(\log V_{max})$ fixed-threshold detection calls. If each detection call evaluates a tree of size T_{τ_i} , bounded by Montanaro’s primitive taking $O(\sqrt{T_{\tau_i}} m \log m)$ queries, the total implemented algorithm query bound is*

$$O\left(\sqrt{T_{max}} m \log m \log V_{max}\right) \text{ queries,} \quad (6)$$

where $T_{max} = \max_i T_{\tau_i}$ over the thresholds tested by the classical binary-search outer loop.

Proof. Phase 1 sorts the n items by efficiency in $O(n \log n)$ time. Computing the greedy lower bound and the initial LP upper bound takes $O(n)$. Phase 2 invokes the threshold detection primitive up to $O(\log V_{max})$ times. Montanaro’s fixed-threshold detection result is the primitive used

at each threshold; Theorem 4 is not directly implemented by `quantum_bb_optimize`. Summing at most $O(\log V_{max})$ fixed-threshold detection calls, each bounded by $O(\sqrt{T_{\tau_i}} m \log m)$, gives the stated query complexity. ■

B. Discussion: Structural Interpretation

The experiments suggest that the hybrid algorithm adapts across instance distributions. For uncorrelated uniform instances, classical B&B evaluates relatively few states. In the tested instances, subset-sum capacities generate much larger search spaces. The theorem-backed query bound indicates that, conditional on a large LP-pruned tree, square-root dependence on tree size can reduce query complexity relative to classical traversal of the same tree.

C. Comparison with Prior Work

Table V summarizes the key differences.

TABLE V
COMPARISON OF KNAPSACK/SUBSET-SUM ALGORITHMS

Algorithm	Reported Complexity Measure	Space	Prov.	QRAQM
Horowitz–Sahni	$O(2^{n/2})$	$O(2^{n/2})$	Yes	N/A
Grover	$O(2^{n/2})$	$O(n)$	Yes	No
Bernstein et al.	$O(2^{0.241n})$	$O(2^{0.241n})$	No [†]	Yes
Helm & May	$O(2^{0.226n})$	$O(2^{0.226n})$	No [†]	Yes
Montanaro B&B	$\tilde{O}(\sqrt{T} d)$ queries	$O(\text{poly})$	Yes	No
QTG+AA	$O(\sqrt{N_f})$	$O(\text{poly})$	Yes	No
Ours	$O(\sqrt{T_{max}} m \log m \log V_{max})$	$O(\text{poly})$	Yes	No

[†]Relies on Heuristic 2, shown unsound by [6].

T : full B&B tree. T_{LP} : LP-pruned tree ($T_{LP} \ll T$). N_f : feasible states.

Our algorithm achieves $O(\sqrt{T_{max}} m \log m \log V_{max})$ queries, combining the structural advantage of exact LP reduced-cost fixing with composed fixed-threshold quantum detection.

VI. EXPERIMENTAL RESULTS

A. Experimental Setup

We implemented and validated all three phases of Algorithm 4 in Python. Crucially, unlike a complexity-counting argument, the explicit Belovs–Montanaro walk operator is constructed computationally for completed trees and spectral/detection conditions are evaluated. The full optimization algorithm `quantum_bb_optimize` uses a classical binary-search outer loop over thresholds τ_i , invoking the Montanaro detection primitive at each threshold. Therefore, the implementation does not directly implement Montanaro Theorem 4. Phases 1 and 3 are exact classical preprocessing/reconstruction. All reported optima are verified against an exact dynamic-programming solve.

The spectral audit tests: the effective spectral-gap inequality, marked phase-zero weight, and the analytical phase-estimation acceptance criterion. It does NOT empirically measure the asymptotic optimization query complexity.

Experiments used five standard instance classes from Pisinger’s classification [9]: (1) uncorrelated, (2) weakly correlated, (3) strongly correlated, (4) subset-sum, and (5) inverse strongly correlated. For each class, we tested $n \in \{12, 14, 16, 18, 20, 22, 24\}$ with 20 random seeds per configuration.

B. Variable Fixing Results

Table VI shows the filtering ratio α across instance types.

TABLE VI
AVERAGE CORE RATIO $\alpha = m/n$ BY INSTANCE TYPE

Instance Type	α	$ F_1 /n$	$ F_0 /n$	LP Gap
Uncorrelated	0.32	0.35	0.33	Tight
Weakly correlated	0.76	0.12	0.12	Medium
Strongly correlated	0.95	0.02	0.03	Loose
Subset-sum	1.00	0.00	0.00	Very loose
Inverse strongly	0.81	0.10	0.09	Medium

C. Spectral Validation of the Quantum Walk

Across the 1200 theoretically attempted conditions, 889 successfully completed within the simulable regime ($T_{LP} \leq 2000$). We restrict our spectral conclusions strictly to these 889 simulable trees. The taxonomy is: 489 completed marked, 400 completed unmarked, 74 tree-cap marked, 74 seed-level empty-core conditions (37 bypassed, 37 empty), and 163 immediately pruned unmarked conditions. The selection-bias audit confirms that the pre-eigendecomposition core size m is significantly larger for capped trees (median $m = 16$) than for completed trees (median $m = 10$).

For the completed instances, we provide computational evidence for three primary spectral endpoints, bounded by exact Clopper-Pearson 95% confidence intervals (CI):

- 1) **Unmarked Effective Spectral-Gap Inequality:** satisfied for 400/400 completed unmarked trees (95% CI: [0.9908, 1.0000]).
- 2) **Marked Phase-Zero Weight:** satisfied for 489/489 completed marked trees (95% CI: [0.9925, 1.0000]).
- 3) **Phase-Estimation Separation:** 889/889 analytical detection criterion agreement with known marked/unmarked status (95% CI: [0.9959, 1.0000]).

The analytical phase-estimation acceptance probabilities exhibit strong distributional separation between marked and unmarked trees. We performed an independent-sample Mann-Whitney U test on the 889 completed simulated trees ($T_{LP} \leq 2000$). The marked acceptance probabilities (N=489, median=0.5177, IQR=0.1752) were significantly higher than the unmarked probabilities (N=400, median=0.0311, IQR=0.0999), yielding $U = 195600.0$ and $p = 1.05 \times 10^{-145}$.

This Mann-Whitney test supports distributional separation of the analytical phase-estimation acceptance probabilities in the completed simulated subset. It does not support quantum

speedup, runtime improvement, quantum advantage, or complexity improvement. The Clopper-Pearson intervals describe the observed satisfaction/agreement proportions in the completed simulable subset, and do not prove universal theorem validity.

TABLE VII
TAXONOMY OF SPECTRAL VALIDATION

Category	Count
Completed Marked (Phase Zero Validated)	489
Completed Unmarked (Spectral Gap Validated)	400
Subtotal Simulable Trees	889 / 889
Tree-Cap Marked (Truncated $T_{LP} > 2000$)	74
Unmarked (Immediately Pruned)	163
Seed-Level Empty Core	74
Total Attempted Conditions	1200

Validated against theoretical detection criterion.

D. Classical Runtime and Memory Characterization

We benchmarked the wall-clock execution time and peak memory consumption of classical B&B against LP-pruned classical B&B on a standard Python 3 execution environment. Instrumentation used `time.perf_counter_ns()` and `tracemalloc`. Across 1500 benchmark records at scales $n \in \{12, 14, 16, 18, 20\}$, we performed a paired Wilcoxon signed-rank test on identical instances.

For these extremely small scales, classical B&B without exact LP preprocessing was universally faster in wall-clock time across all classes (paired median runtime ratio $C/LP \approx 0.24$ to 0.62, Wilcoxon $p < 10^{-4}$ for most). This reflects the constant-factor overhead of repeated Dantzig bound evaluations in Python dominating the trivial search space at $n \leq 20$. LP-pruned peak memory remained minuscule (median 1.2–1.8 KB).

The hybrid state-vector simulation runtime is not an estimate of physical quantum execution time and is therefore excluded from claims of wall-clock quantum acceleration. We do not report any wall-clock quantum speedup.

VII. DISCUSSION

A. Instance-Dependent Query-Complexity Behavior

Our results reveal a key structural insight: the sources of search reduction are instance-dependent and complementary, creating a synergistic algorithm that exploits the strengths of both classical preprocessing and quantum search.

For *uncorrelated instances*, the LP relaxation bound is tight (LP gap $\approx O(1)$), enabling aggressive variable fixing ($\alpha \approx 0.32$ at $n = 24$, dropping to median $\alpha = 0.0276$ at $n = 10,000$). The B&B tree is correspondingly small ($T_{LP} \approx 70$ at $n = 24$). Here, most of the benefit comes from Phase 1 (classical), with the quantum phase contributing a modest additional $\sqrt{T_{\max}} m \log m / T_{\max}$ factor in query count.

For *strongly correlated and subset-sum instances*, variable fixing is ineffective ($\alpha \rightarrow 1$) because all item efficiencies are nearly identical. However, the tested strongly correlated and subset-sum instances produced substantially larger LP-pruned trees ($T_{LP} \approx 3079$ at $n = 24$). Here, Phase 2 provides the key query reduction: Montanaro’s quantum walk achieves a quantum query reduction over substantially larger LP-pruned B&B trees.

This synergy is the core novelty: *In the tested instance classes, the relative contribution of LP fixing and the theorem-backed quantum query bound varies substantially with instance structure, and the two phases address different algorithmic bottlenecks.* For uncorrelated instances, $\alpha \ll 1$ makes the quantum phase trivial; For larger LP-pruned trees, the fixed-threshold detection bound has square-root dependence on tree size, with the explicit depth factor m . In both regimes, the algorithm adapts automatically.

B. Hardware Validation Insights

Our IBM `ibm_fez` experiments reveal a clear NISQ noise boundary for amplitude amplification circuits. Table VIII summarizes the measured hardware success probability versus the ideal simulator success probability across scaled core sizes.

TABLE VIII
IBM HARDWARE RESULTS VS. SIMULATOR SUCCESS PROBABILITY

Instance	Sim Prob	HW Prob	Transpiled Depth	2Q Gates
Core-3A (Easy)	0.9453	0.6743	301	78
Core-3B (Medium)	0.9453	0.6671	294	78
Core-4A (Hard)	0.9613	0.1235	2106	661
Core-4B (Subset-Sum)	0.9613	0.1344	2102	659
Core-5A (Uncorrelated)	0.9992	0.0237	12512	4017
Core-5B (Correlated)	0.9613	0.0637	9273	3012
Core-6A (Hard)	0.9992	0.0312	50520	16379
Core-6B (Subset-Sum)	0.9981	0.0330	38129	12456

At $n \leq 4$ (circuit depth $\leq 2,100$, ≤ 661 two-qubit gates), the correct optimal solution emerges as the dominant measurement outcome with success probabilities of 12–67%. At $n \geq 5$ (circuit depth $\geq 9,000$, $\geq 3,000$ two-qubit gates), noise overwhelms the quantum signal, and the output approaches a uniform distribution.

Fidelity was not independently measured. Backend calibration snapshots were not retained as controlled experimental covariates; consequently, execution-time T1, T2, readout-error, and two-qubit-error statistics are not used for quantitative attribution of the observed hardware–simulator discrepancy.

This characterization has practical implications. For our hybrid algorithm, the quantum phase operates on the *core* items only ($m = \alpha n$). Since $\alpha \ll 1$ for uncorrelated and weakly correlated instances, LP fixing reduces the number of variables exposed to the quantum phase, but the present hardware experiments support only very small cores. No claim is made that current hardware can execute the full quantum B&B phase for large reduced cores.

C. Limitations

Hardware requirements. The fixed-threshold detection primitive has an $O(\sqrt{T_\tau} m \log m)$ query bound under the Montanaro oracle-model assumptions, each involving an LP bounding oracle with reversible marked and threshold predicates. For large core sizes ($m \geq 5$), this demands fault-tolerant quantum hardware, as our IBM experiments demonstrate.

NISQ noise boundary. Our hardware experiments show that current Heron r2 processors can reliably execute amplitude amplification only for cores with $n \leq 4$ using our unitary oracle construction. Decomposed gate-level oracles and error mitigation techniques [35] may extend this boundary.

Small- n overhead. At $n \leq 24$, the Montanaro $\text{poly}(d)$ overhead can make the quantum algorithm slower than classical B&B for easy instances. The quantum query reduction is asymptotic and becomes pronounced as n grows and the LP-pruned tree becomes sufficiently large.

1) *Resource-Estimation Crossover Analysis:* This analysis compares the quantum T-gate cost of the composed query bound against the classical per-node operation cost to derive the crossover threshold. This is strictly an operation-count comparison, not a wall-clock time estimate. Table IX details the combined per-query T-cost (C_Q), aggregating the resources for predicate V (Table III) and the LP heuristic h (Table II). We explicitly note that these counts constitute a partial lower bound because the cited source for the VBE multiplier does not supply an exact gate-count formula [39].

TABLE IX
PER-QUERY T-COST ESTIMATES (C_Q) AND REQUIRED CROSSOVER THRESHOLDS ($T_{\max}$)

Core Size (m)	C_Q (Partial Lower Bound)	Required $T_{\max}$	Crossover
3	$\geq 196,468$	$\geq 3.95 \times 10^{13}$	
4	$\geq 240,986$	$\geq 3.49 \times 10^{14}$	
5	$\geq 269,760$	$\geq 1.50 \times 10^{15}$	

Setting the classical cost ($T_{\max}$) equal to the quantum cost gives the crossover condition: $T_{\max} \geq (C_Q \cdot m \log_2 m \cdot \log_2 V_{\max})^2$. An adversarial comparison against the empirical data reveals that none of the tested instances approach this required crossover threshold by many orders of magnitude; the maximum observed tree sizes ($T_{LP} \approx 70$ for uncorrelated and $T_{LP} \approx 3,079$ for strongly correlated instances at $n = 24$) are ten to eleven orders of magnitude too small. Because the T-cost estimate is a strict lower bound, the true crossover point can only be larger, strengthening the conclusion that the overhead of reversible arithmetic overwhelms the quantum query reduction for the evaluated regime.

Pseudo-polynomial regime. For instances with small integer coefficients, dynamic programming runs in $O(nW)$ time and space, and dominates: our theoretical comparison to branch-and-bound is meaningful only in the large-coefficient regime where DP is infeasible due to time and memory limits.

We make this scope explicit rather than claiming general advantage over all classical methods.

Detection reliability and simulability. Our phase-estimation precision constant is calibrated empirically. Walk simulation restricts the spectral audit to trees with $T_{LP} \leq 2000$ nodes. **Oracle Completeness.** Proposition 1 implements the reversible evaluation/threshold predicate V . For the LP bounding heuristic h , we have specified and verified an exact fixed-precision arithmetic formula (Lemma V-A) and established the complete register layout and qubit resource estimates (Table II). However, full gate-level decomposition of the required arithmetic blocks (multipliers and dividers) remains deferred. Consequently, while the mathematical exactness and qubit overhead of h are confirmed, we still do not claim a full-algorithm gate-complexity bound.

VIII. CONCLUSION

We presented a hybrid algorithm that couples LP-guided reduced-cost variable fixing with Montanaro’s quantum branch-and-bound for the 0/1 knapsack problem—to our knowledge among the first to combine provability, gate-model feasibility, knapsack-specific LP fixing, instance adaptivity, and computational spectral validation of the explicit walk operator (Table I). The implemented threshold-search composition has query complexity $O(\sqrt{T_{\max}} m \log m \log V_{\max})$, where $T_{\max}$ is the maximum LP-pruned tree size encountered and $m = \alpha n$ is the core size after variable fixing. We established two key results:

- 1) *Correctness*: reduced-cost fixing is exact—no heuristic assumptions are involved—and Montanaro’s quantum walk finds the optimal B&B solution with high probability.
- 2) *Instance-dependent synergy*: for uncorrelated instances, LP fixing achieves median $\alpha = 0.0276$ at scale ($n = 10,000$), reducing the effective search to a trivial core; for the tested strongly constrained instances with substantially larger LP-pruned trees, the theorem-backed fixed-tree detection primitive has square-root dependence on tree size, up to the stated depth factors, relative to classical traversal of the same tree.

Experiments on 700 instances across five Pisinger difficulty classes confirmed 100% correctness against exact dynamic programming. Critically, a computational simulation of the quantum walk tested the underlying effective spectral-gap inequality and phase-zero weight constraints across 889 completed simulated trees ($T_{LP} \leq 2000$), providing computational evidence that the implemented LP-pruned walk satisfies the tested spectral conditions underlying Montanaro’s detection analysis. A fault-tolerant resource estimate for our answer-agnostic reversible oracle establishes baseline component costs for the predicate V , and a NISQ characterization on IBM’s `ibm_fez` processor (156-qubit Heron r2) identified the correct solution as the top-1 measured state for $n \leq 4$ while delineating the noise boundary at $n \geq 5$.

Future work includes: (a) extending the analysis to multi-dimensional and multi-objective knapsack variants; (b) imple-

menting decomposed gate-level oracles to reduce circuit depth and extend the hardware-viable range beyond $n = 4$ core items; (c) integrating quantum error mitigation techniques to improve hardware success probabilities at larger core sizes; (d) deriving tighter analytical bounds on T_{LP} as a function of instance parameters such as the LP integrality gap; and (e) evaluating the algorithm on next-generation fault-tolerant quantum processors as they become available.

IX. DATA AVAILABILITY AND REPRODUCIBILITY

To ensure full reproducibility and scientific transparency, the complete source code for our classical B&B, quantum walk simulation, oracle compilation, and IBM Quantum hardware experiments is publicly available. The repository includes scripts to regenerate the 700 Pisinger instances, verify all optimizations against dynamic programming, and reproduce all tables and figures. Crucially, the repository simulates the exact Belovs–Montanaro walk operator computationally to test spectral conditions, moving beyond theoretical complexity assertions.

REFERENCES

- [1] E. Horowitz and S. Sahni, “Computing partitions with applications to the knapsack problem,” *Journal of the ACM*, vol. 21, no. 2, pp. 277–292, 1974.
- [2] L. K. Grover, “A fast quantum mechanical algorithm for database search,” in *Proc. 28th Annual ACM Symposium on Theory of Computing (STOC)*, 1996, pp. 212–219.
- [3] G. Brassard, P. Høyer, M. Mosca, and A. Tapp, “Quantum amplitude amplification and estimation,” *Contemporary Mathematics*, vol. 305, pp. 53–74, 2002.
- [4] D. J. Bernstein, S. Jeffery, T. Lange, and A. Meurer, “Quantum algorithms for the subset-sum problem,” in *Post-Quantum Cryptography (PQCrypto)*, 2013, pp. 16–33.
- [5] A. Helm and A. May, “Subset sum quantumly in 1.17^n ,” in *Proc. Theory of Quantum Computation, Communication and Cryptography (TQC)*, 2018.
- [6] X. Bonnetain, R. Bricout, A. Schrottenloher, and Y. Shen, “Improved classical and quantum algorithms for subset-sum,” in *Advances in Cryptology – EUROCRYPT*, 2020, pp. 633–660.
- [7] S. Jaques and A. G. Rattew, “Qram: A survey and critique,” *arXiv:2305.10310*, 2023.
- [8] G. B. Dantzig, “Discrete-variable extremum problems,” *Operations Research*, vol. 5, no. 2, pp. 266–277, 1957.
- [9] H. Kellerer, U. Pferschy, and D. Pisinger, *Knapsack Problems*. Berlin, Germany: Springer, 2004.
- [10] D. Pisinger, “Core problems in knapsack algorithms,” *Operations Research*, vol. 47, no. 4, pp. 570–575, 1999.
- [11] A. Becker, J.-S. Coron, and A. Joux, “Improved generic algorithms for hard knapsacks,” in *Advances in Cryptology – EUROCRYPT*, 2011, pp. 364–385.
- [12] A. Helm and A. May, “Subset sum algorithms: improved classical and quantum algorithms,” in *Proc. European Symposium on Algorithms (ESA)*, 2021.
- [13] G. Plateau and M. Elkihel, “Analysis and optimisation of the packing tree search algorithm for the knapsack problem,” *European Journal of Operational Research*, vol. 19, no. 2, pp. 211–222, 1985.
- [14] G.-L. Long, “Grover algorithm with zero theoretical failure rate,” *Physical Review A*, vol. 64, no. 2, 2001.
- [15] D. Bulger, W. Baritomp, and G. Wood, “Implementing pure adaptive search with grover’s quantum algorithm,” *Journal of Optimization Theory and Applications*, vol. 116, no. 3, pp. 517–529, 2003.
- [16] C. Dürr and P. Høyer, “A quantum algorithm for finding the minimum,” *arXiv:quant-ph/9607014*, 1996.
- [17] C. H. Bennett, E. Bernstein, G. Brassard, and U. Vazirani, “Strengths and weaknesses of quantum computing,” *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1510–1523, 1997.

- [18] A. Ambainis, "Quantum lower bounds by quantum arguments," *Journal of Computer and System Sciences*, vol. 64, no. 4, pp. 750–767, 2002.
- [19] —, "Quantum walk algorithm for element distinctness," in *Proc. IEEE FOCS*, 2004, pp. 22–31.
- [20] F. Magniez, A. Nayak, J. Roland, and M. Santha, "Search via quantum walk," *SIAM Journal on Computing*, vol. 40, no. 1, pp. 142–164, 2011.
- [21] S. Jaques and J. M. Schanck, "Quantum lattice sieving with reduced memory requirements," in *Advances in Cryptology – EUROCRYPT*, 2020.
- [22] J. Shi, "A quantum algorithm for the solution of the 0-1 knapsack problem," *arXiv:2006.09838*, 2020.
- [23] V. Dunjko, Y. Ge, and J. I. Cirac, "Computational speedups using small quantum devices," *Physical Review Letters*, vol. 121, no. 25, 2018.
- [24] C. Patvardhan, S. Bansal, and A. Srivastav, "A quantum search method for quadratic and multidimensional knapsack problems," *Journal of Theoretical and Applied Information Technology*, vol. 71, no. 3, 2015.
- [25] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," *arXiv:1411.4028*, 2014.
- [26] A. Gilliam, S. Woerner, and C. Goniculea, "Grover adaptive search for constrained polynomial binary optimization," *arXiv:1912.04088*, 2019.
- [27] A. Ajagekar and F. You, "Quantum computing assisted optimization for energy systems," *Computers & Chemical Engineering*, vol. 143, 2020.
- [28] H. B. Norambuena *et al.*, "Hybrid quantum genetic algorithm for the 0-1 knapsack problem," *Expert Systems with Applications*, vol. 223, 2023.
- [29] X. Geng *et al.*, "Quantum-inspired wolf pack algorithm to solve the 0-1 knapsack problem," *Mathematical Problems in Engineering*, 2020.
- [30] S. Li *et al.*, "A binary multi-scale quantum harmonic oscillator algorithm for 0-1 knapsack problem," *IEEE Access*, vol. 8, pp. 19 946–19 962, 2020.
- [31] A. Montanaro, "Quantum speedup of branch-and-bound algorithms," *Physical Review Research*, vol. 2, no. 1, 2020.
- [32] —, "Quantum-walk speedup of backtracking algorithms," *Theory of Computing*, vol. 14, no. 15, pp. 1–24, 2018.
- [33] C. Sanavio, P. Klös, and M. Schwarz, "Hybrid classical-quantum branch-and-bound algorithm for solving integer linear programs," *arXiv:2403.09560*, 2024.
- [34] Y. Ding *et al.*, "Quantum tree generator improves qaoa state-of-the-art for the knapsack problem," *arXiv:2311.04657*, 2023.
- [35] J. Preskill, "Quantum computing in the nisq era and beyond," *Quantum*, vol. 2, p. 79, 2018.
- [36] S. Bravyi, D. Gosset, and R. König, "Quantum advantage with shallow circuits," *Science*, vol. 362, no. 6412, pp. 308–311, 2018.
- [37] Qiskit Development Team, "Qiskit: An open-source framework for quantum computing," <https://qiskit.org>, 2023.
- [38] H. Thapliyal, T. S. S. Varun, and E. Munoz-Coreas, "Quantum circuit design of integer division optimizing ancillary qubits and t-count," *arXiv:1609.01241*, 2016.
- [39] V. Vedral, A. Barenco, and A. Ekert, "Quantum networks for elementary arithmetic operations," *Physical Review A*, vol. 54, no. 1, pp. 147–153, 1996.
- [40] T. G. Draper, "Addition on a quantum computer," *arXiv:quant-ph/0008033*, 2000.